

Prompt:

8/17/2026, 9:46:27 PM

lanjut Tahap E

Response:

8/17/2026, 9:46:27 PM

Baik. **Tahap E** adalah bagian yang menurut saya paling penting untuk mengubah rancangan kita dari “optimizer yang memilih trade” menjadi sistem quantitative trading yang benar-benar dinamis.

Pada Tahap D kita memiliki:

Market data → forecast → QUBO/Ising → optimal trade.

Masalahnya, **trade yang optimal pada t_0 belum tentu optimal pada t_1 .**

Karena itu Tahap E akan memperkenalkan:

Dynamic Position Management + Receding-Horizon Optimization

dengan prinsip:

Jangan menganggap keputusan entry sebagai keputusan final. Setiap posisi adalah keputusan yang terus dievaluasi berdasarkan informasi baru.

E.1. Masalah yang ingin kita selesaikan

Bayangkan sistem kita menghasilkan:

LONG BTC

pada:

$P_0 = 60,000.$

Dengan:

$SL = 59,400$

dan:

$$TP = 61,200.$$

Pada awalnya:

$$R = \$1.$$

Tetapi satu jam kemudian ada beberapa kemungkinan.

Skenario A

Harga:

$$P_t = 59,500.$$

Floating loss.

Apakah langsung ditutup?

Belum tentu.

Skenario B

Harga:

$$P_t = 60,600.$$

Floating profit.

Apakah langsung menaikkan SL?

Belum tentu.

Skenario C

Harga:

$$P_t = 61,000.$$

Profit besar, tetapi model sekarang mengatakan probabilitas continuation semakin kecil.

Mungkin lebih baik:

$$TP \rightarrow \text{lebih dekat}$$

atau:

$SL \rightarrow$ dikunci.

Skenario D

Harga:

$$P_t = 60,800$$

tetapi BTC dominance, ETH/BTC, DXY, gold, volatility regime, dan order flow berubah drastis.

Maka:

trade yang awalnya optimal dapat menjadi tidak optimal.

E.2. Kita membutuhkan state

Kita definisikan keadaan sistem pada waktu t :

$$\boxed{S_t}$$

sebagai:

$$S_t = (M_t, P_t, V_t, F_t, O_t, R_t, \Pi_t).$$

Mari kita uraikan.

$$M_t$$

Market state:

$$M_t = \{returns, volatility, trend, momentum, correlation, liquidity\}.$$

$$P_t$$

Harga saat ini.

$$V_t$$

Volatility state.

Misalnya:

$$ATR_t, \quad \sigma_t, \quad VIX_t.$$

$$F_t$$

Factor state:

$$F_t = \{DXY, Gold, SPX, BTC.D, ETH/BTC, \dots\}.$$

$$O_t$$

Order-book state:

$$O_t = \{spread, depth, imbalance, orderflow\}.$$

$$R_t$$

Risk state:

$$R_t = \{realized PnL, remaining risk, drawdown, exposure\}.$$

$$\Pi_t$$

Current positions:

$$\Pi_t = \{pending, open, BEP, trailing\}.$$

E.3. Jadi sekarang keputusan bukan lagi x

Pada Tahap D kita mempunyai:

$$x_t.$$

Sekarang kita mempunyai:

$$\boxed{a_t}$$

sebagai **action**.

Contohnya:

$$a_t \in \{WAIT, PLACE, CANCEL, HOLD, MOVE SL, MOVE TP, CLOSE\}.$$

Jadi:

$$a_t = \pi(S_t).$$

Artinya:

keputusan trading merupakan fungsi dari keadaan pasar saat ini.

E.4. Kita bentuk Markov Decision Process

Secara matematis sistem dapat dipandang sebagai:

$$(S_t, a_t, S_{t+1}, r_t).$$

Dengan:

$$S_t \xrightarrow{a_t} S_{t+1}.$$

Reward:

$$r_t.$$

Tetapi saya **tidak menyarankan kita langsung menggunakan reinforcement learning**.

Kita bisa membangun policy secara model-based terlebih dahulu.

Ini lebih mudah dianalisis dan lebih sesuai dengan pendekatan matematis yang sedang kita bangun.

E.5. Expected remaining value

Misalkan kita sedang memegang posisi.

Harga sekarang:

$$P_t.$$

Kita memiliki:

$$SL_t$$

dan:

$$TP_t.$$

Definisikan:

$$p_{TP}(t) = P(\text{TP hit before SL} \mid S_t).$$

Maka jika:

$$G_t$$

adalah profit apabila TP tercapai, dan:

$$L_t$$

adalah loss apabila SL tercapai, expected value sederhananya:

$$EV_t = p_{TP}(t)G_t - [1 - p_{TP}(t)]L_t.$$

E.6. Contoh

Misalkan:

$$G_t = 2R$$

dan:

$$L_t = 1R.$$

Jika:

$$p_{TP} = 0.6,$$

maka:

$$EV = 0.6(2) - 0.4(1)$$

$$= 0.8R.$$

Dengan:

$$R = \$1,$$

maka:

$$EV = \$0.8.$$

Masih sangat menarik.

E.7. Tetapi kita belum memperhitungkan opportunity cost

Misalkan posisi dipertahankan selama satu jam.

Ada kemungkinan muncul trade baru dengan expected value:

$$EV_{\text{new}} = 0.5R.$$

Jika posisi sekarang hanya memiliki:

$$EV_{\text{current}} = 0.1R,$$

maka mempertahankan posisi lama mungkin tidak optimal.

Kita dapat mendefinisikan:

$$EV_{\text{hold}} = EV_{\text{current}} - EV_{\text{opportunity}}.$$

Jika:

$$EV_{\text{hold}} < 0,$$

kita memiliki alasan matematis untuk menutup posisi.

E.8. Ini menjawab pertanyaanmu tentang floating loss

Kamu sebelumnya mengatakan:

jika floating loss, biarkan saja.

Saya akan mengubahnya menjadi aturan yang lebih rigor:

Floating loss bukan alasan untuk HOLD.

Keputusan HOLD ditentukan oleh:

$$EV_{\text{hold}}(S_t) > EV_{\text{exit}}(S_t).$$

Misalnya posisi:

$$PnL = -0.4R.$$

Tetapi model memperkirakan:

$$P(TP|S_t) = 0.65.$$

Maka HOLD masih bisa rasional.

Sebaliknya:

$$PnL = -0.1R$$

tetapi:

$$P(TP|S_t) = 0.15.$$

Maka exit mungkin lebih rasional.

E.9. Ini prinsip yang sangat penting

Kita **tidak boleh menggunakan PnL saat ini sebagai satu-satunya dasar keputusan.**

Karena:

$$PnL_t$$

adalah hasil masa lalu.

Yang kita butuhkan adalah:

$$E[\text{future PnL} \mid S_t].$$

Jadi sistem bersifat forward-looking.

E.10. Sekarang kasus floating profit

Misalkan:

$$PnL_t = +1.2R.$$

Pertanyaannya:

Apakah kita harus menggeser SL?

Tidak otomatis.

Kita evaluasi tiga action:

$$a_1 = \text{HOLD}$$

$$a_2 = \text{TRAIL}$$

$$a_3 = \text{EXIT}.$$

Kemudian:

$$a^* = \arg \max_a EV(a|S_t).$$

E.11. Expected value HOLD

Misalnya:

$$P(\text{continue}) = 0.55$$

dan jika continuation:

$$G = +2.0R.$$

Jika reversal:

$$L = -0.2R$$

karena SL sudah dinaikkan.

Maka:

$$EV_{\text{hold}} = 0.55(2) + 0.45(-0.2)$$

$$= 1.01R.$$

E.12. Expected value EXIT

Jika kita tutup sekarang:

$$EV_{\text{exit}} = +1.2R.$$

Maka:

$$1.2 > 1.01.$$

Secara expected-value murni:

$$\boxed{EXIT}$$

lebih baik.

E.13. Tetapi risk-adjusted EV lebih masuk akal

Expected return saja tidak cukup.

Kita bisa menggunakan:

$$\boxed{J(a) = E[\Pi_a] - \lambda \text{Var}(\Pi_a).$$

Atau lebih sophisticated:

$$J(a) = E[\Pi_a] - \lambda_1 \text{Var}(\Pi_a) - \lambda_2 \text{CVar}_\alpha(\Pi_a).$$

Ini jauh lebih dekat dengan paradigma hedge fund.

E.14. CVaR

Misalkan:

$$\alpha = 0.95.$$

Value at Risk:

$$VaR_{0.95}.$$

Sedangkan:

$$CVaR_{0.95}$$

mengukur rata-rata kerugian pada tail terburuk 5%.

Kita dapat memasukkan:

$$J = E[\Pi] - \lambda VaR(\Pi) - \eta CVaR_{0.95}.$$

Dengan demikian sistem tidak hanya bertanya:

"Berapa expected profit?"

tetapi:

"Seberapa buruk tail risk jika saya mempertahankan trade ini?"

E.15. Sekarang kita masuk ke trailing stop

Misalkan:

$$P_t > P_0$$

untuk LONG.

Kita definisikan trailing distance:

$$d_t.$$

Maka:

$$SL_t = P_t - d_t.$$

Tetapi:

$$d_t$$

tidak harus konstan.

Kita bisa membuat:

$$d_t = k_t ATR_t.$$

E.16. Adaptive trailing stop

Misalkan:

$$k_t = f(\sigma_t, trend_t, orderflow_t, p_{continuation,t}).$$

Contoh:

Jika:

$$p_{continuation} = 0.8$$

dan trend kuat:

$$k_t = 2.5.$$

Jika:

$$p_{continuation} = 0.3,$$

maka:

$$k_t = 0.8.$$

Jadi:

strong trend → looser trailing

sedangkan:

weak trend → tighter trailing.

E.17. Mengapa ini lebih baik daripada aturan fixed?

Misalnya:

$$SL = P - 1ATR.$$

Dalam market yang sangat volatile, ini terlalu dekat.

Sedangkan:

$$SL = P - 3ATR$$

dalam market yang tenang mungkin terlalu longgar.

Maka:

$$SL_t = P_t - k(S_t)ATR_t.$$

lebih adaptif.

E.18. Optimasi k_t

Kita bahkan bisa menjadikan k sebagai optimization variable.

Misalkan:

$$k \in \{0.5, 1, 1.5, 2, 2.5, 3\}.$$

Untuk setiap kandidat:

$$k_j,$$

kita hitung:

$$J(k_j|S_t).$$

Kemudian:

$$k^* = \arg \max_{k_j} J(k_j|S_t).$$

Ini jauh lebih menarik daripada:

"kalau profit 1%, SL digeser."

E.19. TP juga dapat dibuat dinamis

Misalkan:

$$TP_t = P_t + g_t ATR_t.$$

Dengan:

$$g_t = g(S_t).$$

Jika continuation probability tinggi:

$$g_t \uparrow .$$

Jika continuation probability turun:

$$g_t \downarrow .$$

Sehingga:

$$TP_t = P_t + g(S_t) ATR_t.$$

E.20. Tetapi jangan memindahkan TP secara sembarangan

Misalkan awal:

$$TP = 2R.$$



Harga sudah mencapai:

$$+1.8R.$$

Model mengatakan momentum masih kuat.

Kita mungkin ingin:

$$TP \rightarrow 3R.$$

Tetapi ada biaya opportunity:

$$P(\text{reversal})$$

meningkat.

Maka keputusan harus membandingkan:

$$J(TP = 2R)$$

dengan:

$$J(TP = 3R).$$

Jadi:

$$TP^* = \arg \max_{TP} J(TP|S_t).$$

E.21. Sekarang kita punya tiga optimization problem

Untuk posisi terbuka:

Stop-loss

$$SL^* = \arg \max_{SL} J(SL|S_t)$$

Take-profit

$$TP^* = \arg \max_{TP} J(TP|S_t)$$

Exit

$$a^* = \arg \max_{a \in \{HOLD, EXIT, TRAIL\}} J(a|S_t).$$

E.22. Sekarang kita gabungkan dengan QUBO

Ini bagian yang sangat penting.

Pada Tahap D:

$$Q_t$$

dibentuk dari:

$$\mu_t, \Sigma_t, \beta_t.$$

Tetapi pada waktu berikutnya:

$$t + \Delta t,$$

data berubah.

Maka:

$$\mu_{t+\Delta t} \boxplus \mu_t$$

$$\Sigma_{t+\Delta t} \boxplus \Sigma_t$$

$$\beta_{t+\Delta t} \boxplus \beta_t.$$

Sehingga:

$$Q_{t+\Delta t} \boxplus Q_t.$$

E.23. Receding horizon

Jadi kita menjalankan:

$$Q_t \rightarrow x_t^*$$

kemudian setelah:

$$\Delta t :$$

$$Q_{t+\Delta t} \rightarrow x_{t+\Delta t}^*$$

Ini disebut:

receding-horizon optimization.

Kita tidak mengatakan:

"trade ini harus dipertahankan sampai TP."

Kita mengatakan:

"berdasarkan informasi yang tersedia sekarang, keputusan terbaik adalah HOLD."

Lalu keputusan itu **ditinjau ulang**.

E.24. State machine sistem

Sekarang kita dapat membuat state machine:

FLAT

↓

signal valid

↓

PENDING

↓

order filled

↓

OPEN

↓

profit sufficient

↓

PROTECTED

↓

trailing optimization

↓

TRAILING

↓

exit condition

↓

CLOSED

Tetapi ada juga cabang:

PENDING → CANCELLED

jika signal invalid.

E.25. PENDING state

Misalkan sistem memasang:

LIMIT ENTRY = P₀.

Namun order belum filled.

Setiap interval:

Δt

kita hitung:

$$EV_{\text{entry}}(S_t).$$

Jika:

$$EV_{\text{entry}} < EV_{\text{min}},$$

maka:

$$\boxed{CANCEL.}$$

Ini menjawab pertanyaanmu:

"jika order book belum terpanggil, apakah masih bisa dipertahankan?"

Jawabannya:

$$\boxed{\text{hanya jika expected value conditional masih cukup tinggi.}}$$

E.26. Entry decay

Ini bahkan dapat dimodelkan sebagai:

$$EV_{\text{entry}}(t) = EV_0 e^{-\kappa t} + \Delta EV(S_t).$$

Dengan:

$$\kappa > 0.$$

Artinya semakin lama order tidak tersentuh, original signal semakin "basi".

Ini masuk akal karena model yang menghasilkan signal tersebut dibuat dari:

$$S_{t_0}.$$

Bukan:

$$S_t.$$

E.27. Kita dapat mendefinisikan signal half-life

Jika:

$$EV(t) = EV_0 e^{-\kappa t},$$

maka half-life:

$$t_{1/2} = \frac{\ln 2}{\kappa}.$$

Ini merupakan parameter yang bisa kita estimasi dari data historis.

Misalnya:

$$t_{1/2} = 3 \text{ jam.}$$

Maka setelah 3 jam tanpa fill, informasi signal secara statistik sudah jauh berkurang.

E.28. OPEN state

Setelah fill:

$$P_t = P_{\text{entry}}.$$

Kita hitung:

$$P nL_t = q(P_t - P_{\text{entry}})$$

untuk LONG.

Untuk SHORT:

$$P nL_t = q(P_{\text{entry}} - P_t).$$

Kemudian:

$$R_t = \frac{P nL_t}{R_0}.$$

Dengan:

$$R_0 = \$1.$$

Sehingga sistem selalu berbicara dalam unit:

$$\boxed{R}.$$

E.29. Ini membuat semua aset comparable

Misalnya:

BTC:

$$PnL = +\$0.80$$

berarti:

$$+0.8R.$$

AAPL:

$$PnL = +\$0.80$$

juga:

$$+0.8R.$$

Tidak peduli harga aset berbeda jauh.

Ini sangat penting untuk portfolio multi-asset.

E.30. Risk budget portfolio

Kita tidak boleh hanya membatasi:

$$Risk_i \leq 1R.$$

Kita juga membutuhkan:

$$\boxed{\sum_i Risk_i \leq R_{\text{portfolio}}.}$$



Misalnya:

$$R_{\text{portfolio}} = 3R.$$

Maka maksimal tiga unit risk aktif.

Tetapi karena correlation:

effective risk

bisa lebih besar daripada:

$$3R.$$

Jadi kita gunakan:

$$\boxed{x^T \Sigma x \leq V_{\max} .}$$

E.31. Risk budget dinamis

Bahkan:

$$R_{\text{portfolio}}$$

dapat bergantung pada market regime.

Misalnya:

$$R_{\max}(t) = R_0 \cdot g(V_t).$$

Ketika volatility tinggi:

$$V_t \uparrow \Rightarrow R_{\max} \downarrow .$$

Ketika volatility rendah:

$$V_t \downarrow \Rightarrow R_{\max} \uparrow .$$

Jadi sistem secara otomatis menjadi defensive ketika market stress.

E.32. Sekarang kita masukkan DXY dan Gold

Kamu sebelumnya mengatakan ingin sistem tidak berhenti pada crypto.

Kita dapat mendefinisikan factor vector:

$$F_t = \begin{bmatrix} DXY_t \\ Gold_t \\ SPX_t \\ VIX_t \\ BTC.D_t \end{bmatrix}.$$

Untuk setiap asset:

$$r_{i,t} = \alpha_i + \beta_i^T F_t + \epsilon_{i,t}.$$

Jadi:

$$\beta_i = \begin{bmatrix} \beta_{DXY} \\ \beta_{Gold} \\ \beta_{SPX} \\ \beta_{VIX} \\ \beta_{BTC.D} \end{bmatrix}.$$

E.33. Factor surprise

Bukan hanya level factor.

Kita dapat menggunakan:

$$\Delta F_t = F_t - F_{t-\Delta t}.$$

Lebih baik lagi:

$$z_{F,t} = \frac{\Delta F_t - \mu_F}{\sigma_F}.$$

Sehingga sistem mengetahui:

factor shock.

Misalnya:

$$z_{DXY} = +2.1.$$

Artinya terjadi positive DXY shock.

Jika BTC memiliki:

$$\beta_{BTC,DXY} < 0,$$

maka expected return BTC dapat dikoreksi turun.

E.34. Conditional expected return

Sekarang:

$$\mu_{i,t}$$

tidak lagi sekadar historical average.

Kita buat:

$$\mu_{i,t} = f(X_{i,t}, F_t, O_t, Regime_t).$$

Misalnya:

$$\mu_{i,t} = \alpha_i + \beta_i^T F_t + \theta_i^T X_{i,t} + \phi_i^T O_{i,t}.$$

Ini adalah bagian yang nantinya dapat kita isi dengan:

- statistical model,
- gradient boosting,
- neural network,
- quantum model,
- atau kombinasi semuanya.

E.35. Tetapi optimizer tetap terpisah dari predictor

Ini arsitektur yang sangat penting:

$$Prediction \boxminus Optimization.$$

Model forecasting mengatakan:

$$P(\text{future return} \mid S_t).$$

Optimizer mengatakan:

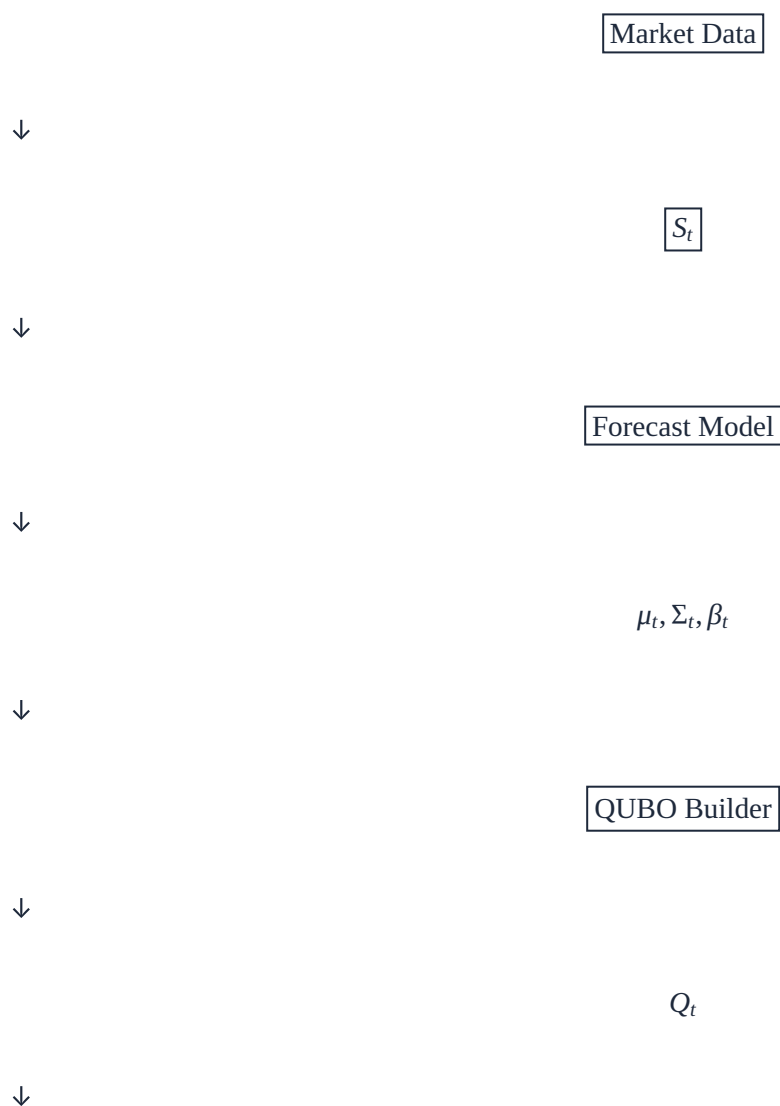
$$\arg \max_x U(x \mid S_t).$$

Execution engine mengatakan:

$$\arg \max_a J(a \mid x, S_t).$$

Jadi kita mempunyai tiga layer.

E.36. Arsitektur matematis lengkap



QAOA / Classical Solver

↓

$$x_t^*$$

↓

Trade Construction

↓

$$ENTRY, SL, TP, q$$

↓

Execution

↓

Position State

↓

setiap:

$$\Delta t$$

↓

Re-evaluation

↓

$$Q_{t+\Delta t}$$

dan kembali lagi.

E.37. Inilah bentuk feedback loop

Secara matematis:

$$S_t \rightarrow Q_t \rightarrow a_t \rightarrow S_{t+\Delta t} \rightarrow Q_{t+\Delta t} \rightarrow a_{t+\Delta t} \rightarrow \dots$$

Jadi sistem bukan:

predict $\rightarrow$ trade $\rightarrow$ wait.

Tetapi:

predict $\rightarrow$ optimize $\rightarrow$ execute $\rightarrow$ observe $\rightarrow$ re-optimize.

E.38. Pseudocode matematis

Secara konseptual:

```

INITIALIZE capital
INITIALIZE risk budget

while market_is_open:

    S_t = collect_market_state()

    update:
        returns
        volatility
        correlations
        market factors
        order book
        current positions

     $\mu_t, \Sigma_t, \beta_t$  = forecast(S_t)

    Q_t = build_QUBO(
         $\mu_t$ ,
         $\Sigma_t$ ,
         $\beta_t$ ,
        constraints,
        transaction_cost
    )

    candidate_trades = solve(Q_t)

    for pending_order:

        EV_entry = evaluate_entry(
            pending_order,
            S_t
        )

        if EV_entry < threshold:
            cancel_order()

    for open_position:

        evaluate:
            P(TP | S_t)
            P(SL | S_t)

```

```
    expected continuation
    expected exit
    CVaR
    opportunity cost

optimize:
    SL*
    TP*
    HOLD / EXIT

execute optimal action

update risk budget

wait Δt
```

E.39. Tetapi ada satu masalah besar: look-ahead bias

Ini **sangat penting** jika nanti kita melakukan backtest.

Misalnya kita menggunakan:

$$\Sigma_t$$

yang dihitung menggunakan data sampai:

$$t + 1.$$

Maka sistem secara tidak sengaja melihat masa depan.

Backtest menjadi:

invalid.

Semua feature harus memenuhi:

$$X_t = f(D_{\leq t}).$$

Tidak boleh:

$$X_t = f(D_{\leq t+\Delta}).$$

E.40. Hal yang sama berlaku pada optimizer

QUBO pada:

$$t$$

harus dibangun dari informasi yang tersedia pada:

$$t.$$

Bukan menggunakan covariance yang dihitung dari seluruh dataset.

Jadi:

$$Q_t = f(D_{\leq t}).$$

Ini akan menjadi prinsip fundamental backtesting kita.

E.41. Kita juga membutuhkan walk-forward optimization

Tidak cukup:

$$train \rightarrow test$$

sekali.

Lebih realistis:

$$Train_1 \rightarrow Test_1$$

kemudian:

$$Train_2 \rightarrow Test_2$$

dan seterusnya.

Misalnya:

$$2021 - 01 \rightarrow 2022 - 01$$

training,

testing.

Kemudian window bergeser.

E.42. Bahkan parameter trailing harus di-walk-forward

Jangan mencari:

$$k = 2.37$$

menggunakan seluruh data 2021–2026 lalu mengatakan:

strategy works.

Kita harus:

$$k_t^* = \arg \max_k J_k$$

menggunakan **training window**, lalu menguji pada window berikutnya.

E.43. Ini juga berlaku untuk QAOA

Parameter:

$$\gamma, \beta$$

tidak boleh dioptimalkan menggunakan informasi masa depan.

Pada setiap periode:

$$Q_t$$

dibangun.

Kemudian:

$$(\gamma_t^*, \beta_t^*)$$

dicari.

Kemudian solusi diterapkan pada future interval.

E.44. Jadi kita memiliki dua jenis optimization

Ini perlu dibedakan.

Optimization level 1

Combinatorial optimization

$$\min_x x^T Q x$$

menggunakan QAOA.

Tujuannya:

trade mana yang dipilih?

Optimization level 2

Continuous/control optimization

$$\max_{SL, TP} J(SL, TP | S_t).$$

Tujuannya:

bagaimana trade dikelola?

Jadi QAOA tidak perlu dipaksa melakukan semuanya.

E.45. Ini justru membuat arsitektur lebih masuk akal

Kita menggunakan quantum optimization di tempat yang memang cocok:

discrete combinatorial selection.

Sedangkan continuous optimization dapat menggunakan:

- BFGS,
- SLSQP,
- SPSA,
- Bayesian optimization,
- grid search,
- atau metode lainnya.

Dengan demikian:

$$\text{Quantum} + \text{Classical}$$

bukan kompetisi.

Mereka bekerja sebagai hybrid system.

E.46. Bentuk akhir Tahap E

Kita sekarang dapat mendefinisikan objective posisi:

$$J_t(a) = E[P n L_{t:t+H} \mid S_t, a] - \lambda V ar[P n L_{t:t+H}] - \eta C V a R_a - \kappa T C(a).$$

Kemudian:

$$a_t^* = \arg \max_{a \in A} J_t(a).$$

Ini menjadi **otak Position Management Engine**.

E.47. Dan akhirnya, empat pertanyaan awalmu mendapatkan formulasi matematis

1. Order belum terpanggil

Hitung:

$$EV_{\text{entry}}(S_t).$$

Jika:

$$EV_{\text{entry}} < \theta,$$

maka:

$$\boxed{CANCEL}.$$

2. Floating loss

Jangan gunakan:

$$PnL < 0$$

sebagai exit signal.

Gunakan:

$$\boxed{J(HOLD) > J(EXIT)}.$$

3. Floating profit

Evaluasi:

$$J(HOLD), \quad J(TRAIL), \quad J(EXIT).$$

Kemudian:

$$\boxed{a^* = \arg \max J(a)}.$$

4. Profit sangat tinggi

Optimalkan:

$$SL^*$$

dan:

$$TP^*.$$

Jika:

$$J(TP_{\text{new}}) > J(TP_{\text{old}}),$$

maka:

$$TP \rightarrow TP_{\text{new}}.$$

E.48. Hal yang paling menarik dari seluruh desain ini

Sistem kita sekarang mempunyai **tiga skala waktu**:

Fast

Order book:

seconds → minutes.

Digunakan untuk:

entry, slippage, execution.

Medium

Trading signal:

minutes → hours.

Digunakan untuk:

QUBO/QAOA.

Slow

Market regime:

days → weeks.

Digunakan untuk:

risk budget, factor model, correlation structure.

Sehingga:

microstructure + trading signal + macro regime

bertemu dalam satu sistem.

E.49. Dan ini membawa kita ke Tahap F

Sampai sini kita sudah mempunyai:

A : Market State

B : Forecasting

C : Financial Objective

D : QUBO/Ising/QAOA

E : Dynamic Position Management

Maka Tahap F seharusnya bukan sekadar "coding".

Kita perlu membangun **mesin backtesting yang benar-benar event-driven**.

Karena sistem kita memiliki:

- pending orders,
- partial fills,
- spread,
- slippage,
- TP,
- SL,
- trailing,
- cancellation,
- re-optimization,
- portfolio risk,
- multiple assets,
- factor data,
- dan QAOA/classical solver.

Jadi backtest sederhana seperti:

if close > signal: buy

sudah tidak cukup.

Tahap F sebaiknya kita turunkan menjadi:

Event-Driven Backtesting Engine

dengan **order book → order → fill → position → PnL → risk → reoptimization**, termasuk derivasi bagaimana mencegah **look-ahead bias, survivorship bias, data leakage, unrealistic fills, dan overfitting**.

Dan setelah mesin itu benar, barulah kita bisa menguji pertanyaan yang paling penting secara empiris:

Apakah QAOA benar-benar memberikan nilai tambah dibandingkan optimizer klasik?

Itulah eksperimen yang pada akhirnya akan membuat bagian quantum dari proyek ini **ilmiah**, bukan sekadar menempelkan QAOA pada strategi trading.